Financial Fraud Detection System

Complete Technical Documentation & User Manual

> **Project Title:** Financial Fraud Detection System & Streamlit Analytics Dashboard
> **Tech Stack:** Python, Streamlit, Scikit-Learn, XGBoost, Pandas, Plotly, NumPy, Joblib
> **Status:** Active — runs locally at http://localhost:8501

Table of Contents

- 1. Executive Summary
- 2. Domain & Problem Statement Theory
- 3. Data Processing & Analytics Methodology
- 4. Feature Engineering
- 5. Machine Learning Algorithms & Mathematical Foundations
- 6. Model Evaluation Metrics & Benchmarks
- 7. System Architecture & Dashboard Modules
- 8. Codebase & Directory Structure
- 9. Installation & Operating Instructions
- 10. Glossary of Terms

1. Executive Summary

The Financial Fraud Detection System is an end-to-end machine learning platform built to identify fraudulent transactions across Credit Card, Debit Card, PayPal, UPI, and NetBanking channels. The system engineers behavioral spending ratios, transaction-timing flags, and cross-border risk indicators, then applies a calibrated classification model to output a fraud probability for every transaction — achieving high recall with zero missed fraud cases on the held-out test set.

2. Domain & Problem Statement Theory

2.1 What Is Financial Fraud Detection?

Financial fraud detection applies statistical methods, data analytics, and machine learning to identify transactions carried out with fraudulent intent — for example, stolen card details, account takeovers, unauthorized identity use, or suspicious cross-border activity.

2.2 Types of Financial Fraud Captured

- Card-Not-Present (CNP) Fraud — online transactions where the physical card is not presented
- Behavioral Anomaly Fraud — sudden, high-value spending that diverges sharply from a customer's historical baseline
- Cross-Border / International Fraud — transactions from high-risk foreign locations with no prior travel notice
- Circadian / Night-Time Exploitation — fraud executed during non-standard hours (roughly 11 PM to 5 AM), when account holders are less likely to verify OTPs promptly
- Suspicious Keyword Fraud — activity flagged through suspicious transaction remarks or communication notes

2.3 Key Data Science Challenges in Fraud Analytics

Extreme Class Imbalance: fraudulent transactions typically make up less than 10% of total volume (9.64% in the training set used here). Standard models biased toward the majority (legitimate) class will under-detect fraud unless explicitly calibrated for imbalance.

Asymmetric Error Costs: a false negative (missed fraud) carries high financial loss, chargebacks, customer dissatisfaction, and regulatory exposure, while a false positive (false alarm) causes only mild friction — a temporary verification step or alert. This asymmetry is why the project prioritizes recall over raw accuracy.

3. Data Processing & Analytics Methodology

3.1 Handling Missing Values (Imputation)

- Numerical features — imputed with the mean or median, depending on skewness
- Categorical features — imputed with the mode (most frequent value)

3.2 Outlier Detection Methods

IQR (Interquartile Range) Method

$$IQR = Q3 - Q1$$

$$Lower\ Bound = Q1 - 1.5 \times IQR \qquad Upper\ Bound = Q3 + 1.5 \times IQR$$

Z-Score Method (Standard Score)

$$Z = (X - \mu) / \sigma$$

Values where |Z| > 3.0 are classified as statistical outliers.

3.3 Feature Scaling — Standardization vs. Normalization

Standardization (StandardScaler)

Transforms a feature to have mean μ = 0 and standard deviation σ = 1. Essential for distance-based and gradient-based algorithms such as Logistic Regression.

$$X\_standardized = (X - \mu) / \sigma$$

Normalization (Min-Max Scaling)

Scales a feature to the range [0, 1]:

$$X\_normalized = (X - X\_min) / (X\_max - X\_min)$$

4. Feature Engineering

Four custom features were engineered to boost model discriminative power:

Feature	Formula / Logic	Purpose
Spend_to_Avg_Ratio	Transaction_Amount / (Average_Spend + 1.0)	Measures how many times a purchase exceeds the customer's historical baseline
Spend_Minus_Avg	Transaction_Amount − Average_Spend	Captures absolute dollar variance from baseline spend
Is_Night_Txn	1 if Hour ≥ 23 or Hour ≤ 5, else 0	Flags circadian / off-hours transaction risk
Suspicious_Keyword_Num	'Yes' → 1, 'No' → 0	Numerical encoding of flagged transaction remarks

5. Machine Learning Algorithms & Mathematical Foundations

5.1 Logistic Regression (Selected Operational Model)

Logistic Regression models the probability P(Y=1|X) using the sigmoid (logistic) function:

$$P(Y=1|X) = \sigma(z) = 1 / (1 + e^{(-z)})$$

where $z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_n X_n$.

Loss Function — Binary Cross-Entropy:

$$J(\beta) = -(1/m)\, \Sigma\, [\, y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})\,]$$

Class Weight Balancing penalizes fraud misclassifications more heavily by scaling class weights in inverse proportion to class frequency.

5.2 Random Forest Classifier

An ensemble technique that combines multiple decision trees using bootstrap aggregation (bagging) and feature subsampling. Splits are chosen to minimize Gini impurity:

$$G = 1 - \Sigma p_i^2$$

This reduces variance and overfitting relative to a single decision tree.

5.3 XGBoost (eXtreme Gradient Boosting)

A gradient-boosted tree ensemble that minimizes an objective combining loss and tree-complexity regularization:

$$L^{(t)} = \Sigma\, l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

$$where\ \Omega(f) = \gamma T + \tfrac{1}{2}\lambda\, \Sigma\, w\_j^2$$

The scale_pos_weight parameter is set to Negative Samples ÷ Positive Samples ≈ 9.36, to balance learning across the imbalanced classes.

6. Model Evaluation Metrics & Benchmarks

6.1 Confusion Matrix Structure

	Predicted Negative (0)	**Predicted Positive (1)**
Actual Negative (0)	True Negative (TN)	False Positive (FP)
Actual Positive (1)	False Negative (FN)	True Positive (TP)

6.2 Mathematical Definitions

Accuracy — (TP + TN) / (TP + TN + FP + FN); can be misleading on imbalanced data.

Precision — TP / (TP + FP); of all flagged transactions, how many were actual fraud.

Recall (Sensitivity) — TP / (TP + FN); of all actual fraud cases, how many were caught.

F1-Score — harmonic mean of precision and recall: 2 × (Precision × Recall) / (Precision + Recall).

ROC-AUC — area under the curve plotting true-positive rate against false-positive rate across all decision thresholds.

6.3 Benchmarks Achieved in This Project

Model	**Accuracy**	**Precision**	**Recall**	**F1-Score**	**ROC-AUC**
Logistic Regression (Selected)	75.40%	28.07%	**100.00%**	0.4384	**0.8933**
Random Forest	77.20%	26.26%	76.04%	0.3904	0.8580
XGBoost	83.30%	28.22%	47.92%	0.3552	0.8665

Conclusion: Logistic Regression achieved 100% recall (zero missed fraud cases) and the highest ROC-AUC (0.8933), making it the optimal choice for fraud-prevention deployment despite its lower raw accuracy relative to XGBoost.

7. System Architecture & Dashboard Modules

```
Dataset (CSV)  ⟶  Feature Engineering  ⟶  Model Benchmarking  ⟶  Streamlit Web
App (app.py)
```

The application (app.py) is structured into five core operational modules:

Executive Dashboard

Real-time KPI metric cards (total volume, fraud rate %, flagged loss) and Plotly charts breaking down fraud by payment method, merchant category, location, and hour of day.

Real-Time Fraud Predictor

A form interface for manual transaction entry that returns a risk score %, a risk badge (High / Moderate / Safe), and an automated natural-language explanation of the contributing risk factors.

Analytics & Insights

Behavioral anomaly scatter plots, device-type risk rates, and suspicious-keyword correlation graphs.

Model Performance

A side-by-side metric comparison table across all three models and a chart of the top 10 most influential features.

Batch CSV Scanner

A drag-and-drop batch transaction scanner with smart auto-column mapping (auto_map_batch_columns) and one-click, downloadable labeled CSV export.

8. Codebase & Directory Structure

```
Fraud_Detection_Zidio/
│
├── train_model.py                        # ML model training script
├── app.py                                # Streamlit web dashboard application
├── PROJECT_DOCUMENTATION.html             # Full documentation webpage (printable to
PDF)
├── PROJECT_DOCUMENTATION.md               # Full markdown documentation file
├── Financial_Fraud_Detection_Project_Report.html  # Executive summary report
│
├── model_artifacts/                       # Model weights & metadata
│   ├── fraud_model.pkl                    # Trained Logistic Regression model
│   ├── scaler.pkl                         # StandardScaler object
│   ├── feature_info.pkl                   # Feature names & metadata
│   └── model_metrics.json                 # Benchmark metrics across models
│
└── Related_files/                         # Datasets & reference notebooks
    ├── financial_fraud_detection_dataset.csv   (5,000 rows)
    └── synthetic_fraud_dataset1.csv             (50,000 rows)
```

9. Installation & Operating Instructions

9.1 Launch the Interactive Dashboard

```
cd path\to\Fraud_Detection_Zidio
streamlit run app.py

# Then open in your browser:
http://localhost:8501
```

9.2 Re-train the Model (Optional)

```
python train_model.py
```

10. Glossary of Terms

- CNP Fraud — Card-Not-Present fraud; fraud committed in transactions where the physical card is absent
- Recall — the proportion of actual fraud cases correctly identified by the model
- Precision — the proportion of flagged transactions that are genuinely fraudulent
- ROC-AUC — a threshold-independent measure of a model's ability to distinguish fraud from legitimate transactions
- Class Imbalance — a dataset condition where one class (legitimate transactions) vastly outnumbers the other (fraud)
- Scale_pos_weight — an XGBoost parameter that rebalances learning focus toward the minority (fraud) class